EQUIJEPA: LEARNING ACTION-CENTERED LATENT TRANSITION GEOMETRY

Anonymous authors
Paper under double-blind review

ABSTRACT

Latent world models learn to predict future states from the current state and an action, and are often also trained to recover the executed action from a pair of consecutive states. However, these methods do not require the same action in different states to produce similar latent transitions, so learned representations need not transfer to new states or add up predictably. We call this missing property *action equivariance*. We introduce EquiJEPA, which jointly learns a latent world model and an effect head that predicts latent transitions from actions alone. Because the head cannot see the state, reducing its loss requires the encoder to make the same action produce similar latent transitions across states, forming a shared action prototype. These shared prototypes can then be applied at new states or added together, with the inverse head translating the resulting transitions into executable actions. We evaluate these operations across four simulated environments, testing whether learned action prototypes can be combined, executed from new states, and reused across command interfaces. In all four, EquiJEPA makes the latent transitions of each action more consistent across states than the base model does, while keeping agent and object state decodable. Its action prototypes also execute more accurately at new states, compose into unseen action combinations, and carry over to a new command interface.

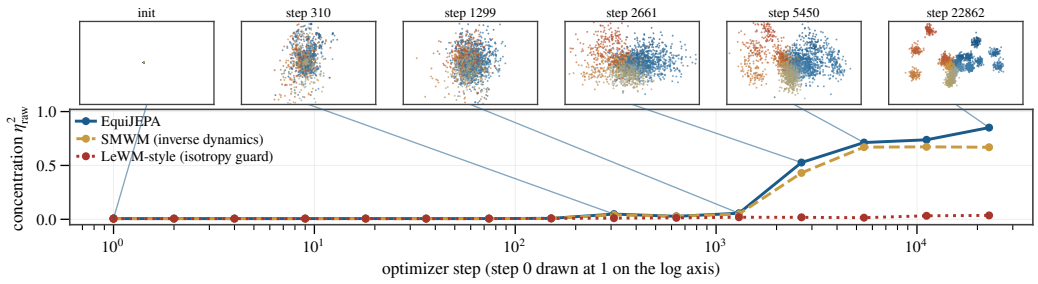


Figure 1: **EquiJEPA learns more consistent latent transitions for the same action across states.** Push-T environment. Top: EquiJEPA's latent transitions at selected checkpoints, projected onto the first two principal components of its final checkpoint and coloured by action direction. Bottom: transition concentration η^2_{raw} (Section 3.3). Inverse dynamics raises concentration, EquiJEPA's effect head raises it further, and the LeWM-style baseline stays low. Protocol in Appendix A.2.

1 INTRODUCTION

A central challenge in embodied AI is to learn representations that capture both the state of the world and how actions transform it (LeCun, 2022). Latent world models encode observations into compact latents and predict the next one from the current one and an action, without reconstructing pixels (Zhou et al., 2025; Sobal et al., 2025; Assran et al., 2025). Yet accurate next-state prediction alone places little constraint on how these transitions are organized geometrically in latent space: two states may admit analogous actions and consequences while their latent transitions bear no systematic relation to one another. Such a representation may be sufficient for prediction, but it need

not expose the reusable structure of how actions transform the world. We ask: *Beyond representing states, how should a world model organize the latent transitions produced by actions?*

Human motor learning offers a useful intuition. People do not appear to learn every movement in every state independently. Learning a motor transformation in one setting generalizes systematically to related movements and states, a phenomenon commonly modeled through reusable motor primitives or basis functions (Thoroughman & Shadmehr, 2000; Donchin et al., 2003). More broadly, structural learning suggests that experience across related motor tasks allows people to extract shared, lower-dimensional structure that transfers to novel instances of the same task family (Braun et al., 2009). In this view, the effect of a learned action is represented relative to its effects elsewhere, which supports interpolation and reuse across states. We adopt the analogous principle for latent world models: if the same action produces analogous transformations in different states, their latent transitions should share geometric structure. Methods such as SMWM (Ivashkov et al., 2026) retain action-relevant information by training an inverse head to identify the action that produced an observed transition. Because decodability only requires actions to remain distinguishable, the inverse head can rely on a few latent dimensions while the rest vary with the state, so the latent transitions of one action need not cluster together. Such a representation identifies the action but provides no shared transition to reuse, add, or compare across states.

Motivated by this distinction, we introduce EquiJEPA, which jointly trains an inverse-dynamics JEPA with a state-blind effect head that predicts latent transitions from actions alone. Because the head cannot see the state, lowering its loss requires the encoder to make the state-independent part of each change dominant, so changes produced by the same action concentrate around a shared prototype. The action prototype captures the shared motion, such as the movement of a robot's end effector, while the residual absorbs changes caused by contact with surrounding objects. Together with the inverse head, this gives actions an explicit representation in latent space: the model can predict a latent transition from an action and generate an action from an intended latent transition (Figure 2b). Figure 1 tracks the development of this shared action structure during training on Push-T. Inverse dynamics increases transition concentration, while adding our state-blind effect head increases it further. The snapshots show EquiJEPA's latent transitions grouping by action direction.

We evaluate whether these learned representations can guide motion from new states and produce movements absent from training. On Push-T, reusing action prototypes at new states gives a mean agent error of 2.67 pixels with EquiJEPA, versus 4.02 for the base model. To test composition, we train a model in a two-dot world on movements of one dot at a time. Adding prototypes to move both dots at once succeeds in 40.1% of trials with EquiJEPA, versus 11.5% for the base.

Our contributions are:

1. **Representation — An action-centered latent world model.** We distinguish action decodability from transition concentration and introduce a training objective that explicitly encourages the latter. The resulting representations keep agent and object state decodable.
2. **Composition — Reusable action structure and downstream applications.** The learned action prototypes support composition across actions and time, as well as execution at new states. Sparse dictionary atoms recover action primitives and transfer to a new command interface.

2 Related work

Reconstruction-free world models and planning. JEPAs and related latent predictive models learn dynamics without reconstructing pixels (LeCun, 2022; Zhou et al., 2025; Sobal et al., 2025; Maes et al., 2026b; Assran et al., 2025). Latent-space model predictive control (Zhou et al., 2025; Hansen et al., 2024) can use sampling-based optimizers such as MPPI (Williams et al., 2016). PRISM learns sampling priors (Wang et al., 2026b), INTACT learns a search-free intent-to-action interface (Sun et al., 2026), and AdaJEPA adapts the world model during closed-loop planning (Wang et al., 2026a). These approaches do not explicitly minimize within-action variation of latent transitions, which is the property EquiJEPA trains for.

Latent actions and action-conditioned dynamics. During pretraining without action labels, video-based models infer latent actions from frame transitions, either as discrete codes (Bruce et al., 2024; Schmidt & Jiang, 2024; Ye et al., 2025) or as continuous vectors (Cui et al., 2024). Because

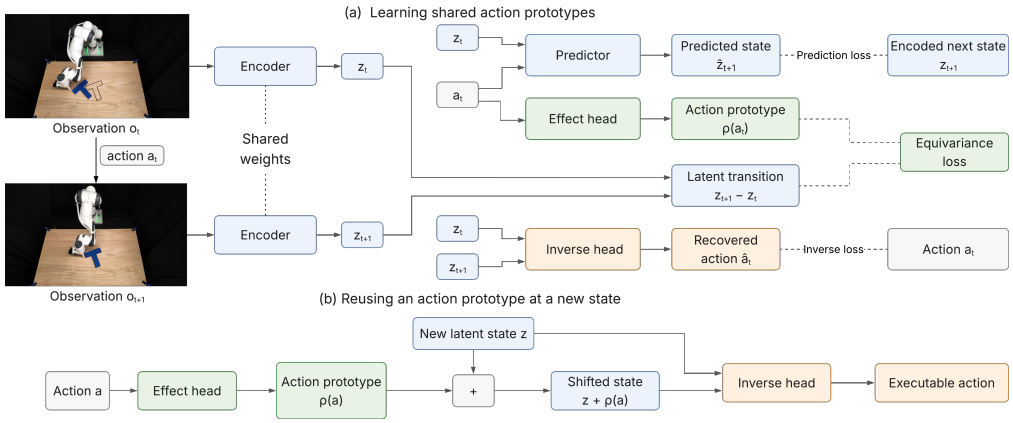


Figure 2: **EquiJEPA training and prototype reuse (schematic).** (a) Training; photos are illustrative only. A shared encoder maps o_t and o_{t+1} to z_t and z_{t+1}. The predictor and inverse head form the base model ($\mathcal{L}_{\text{pred}}$, $\mathcal{L}_{\text{inv}}$). The effect head receives only the action and predicts the action prototype $\rho(a_t)$, which the equivariance loss $\mathcal{L}_{\text{equiv}}$ matches to the latent transition $z_{t+1} - z_t$. All three losses train the encoder. (b) Reusing an action prototype $\rho(a)$ at a new latent state z; the inverse head decodes $(z, z + \rho(a))$ into an executable action (Section 5.1).

these latent actions are not grounded in observed commands, their meaning can differ across states or scenes (Jiang et al., 2026). Earlier structured latent dynamics models constrain transitions to be locally linear (Watter et al., 2015) or factored over objects (Kipf et al., 2020), but the transition still depends on the state. With observed actions, inverse-dynamics training can induce translation-like structure and make actions decodable from latent transitions (Ivashkov et al., 2026). Decodability only requires different actions to remain distinguishable; it does not bound how much the latent transition of one action varies across states (Proposition 1). EquiJEPA trains state-blind action prototypes and measures this within-action variation with controlled replays.

Equivariance in representation learning. Equivariant representation learning maps transformations in the world to predictable transformations in latent space (Higgins et al., 2018; Caselles-Dupré et al., 2019; van der Pol et al., 2020). The homomorphism autoencoder learns one latent transformation per action that applies in every state (Keurti et al., 2023), and seq-JEPA learns invariant and equivariant representations from sequences of transformed views (Ghaemi et al., 2025). These studies evaluate structured transformations rather than contact-rich manipulation, where the effect of a push depends on whether the hand touches the block. EquiJEPA uses a translation $\rho(a)$ as a soft target rather than an exact symmetry, keeps the state-dependent part as an interaction residual, and relies on inverse dynamics to prevent representation collapse.

Sparse dictionaries. After training, we fit a top-k sparse autoencoder (Makhzani & Frey, 2014; Gao et al., 2025) to the frozen latent transitions; it is a downstream tool, not part of the EquiJEPA objective. Top-k autoencoders are a form of sparse coding (Olshausen & Field, 1996) and are used to decompose learned representations (Cunningham et al., 2024). A single-code VQ representation selects one codebook entry (van den Oord et al., 2017), whereas a sparse dictionary represents a transition as a sum of existing atoms. How well this works depends on how concentrated each action's latent transitions are; Section 5 compares the base and EquiJEPA under one dictionary protocol in the two-dot world, Push-T, and OGBench Cube.

3 Method

3.1 Problem formulation

We consider an offline dataset $\mathcal{D}$ of transitions (o_t, a_t, o_{t+1}), where o_t is the current observation, a_t is the executed action, and o_{t+1} is the resulting observation. Our base is an inverse-dynamics JEPA (Ivashkov et al., 2026) (Figure 2a, blue and orange): a shared encoder gives $z_t = h_\psi(o_t)$

and $z_{t+1} = h_\psi(o_{t+1})$, the predictor estimates $\hat{z}_{t+1} = P_\phi(z_t, a_t)$, and the inverse head recovers $\hat{a}_t = g_\omega(z_t, z_{t+1})$. The base model is trained with

$$\mathcal{L}_{\text{pred}} = \|\hat{z}_{t+1} - z_{t+1}\|_2^2, \qquad \mathcal{L}_{\text{inv}} = \|\hat{a}_t - a_t\|_2^2.$$

We call $\Delta z_t = z_{t+1} - z_t$ the *latent transition*, and write $\Delta z(s, a)$ for the latent transition when a is executed from simulator state s. Our goal is to make different executions of the same action share a reusable latent transition without discarding state-dependent outcomes.

3.2 Learning transition concentration

EquiJEPA adds a state-blind effect head $\rho : \mathcal{A} \to \mathbb{R}^d$ (Figure 2a, green). It receives only the action and cannot observe z_t, so it can only capture the part of the latent transition shared across states:

$$\mathcal{L}_{\text{equiv}} = \|\Delta z_t - \rho(a_t)\|_2^2. \tag{1}$$

We call $\rho(a)$ the *action prototype* and the state-dependent remainder the *interaction residual*,

$$r_{\text{act}}(s, a) = \Delta z(s, a) - \rho(a). \tag{2}$$

The residual contains every state-dependent part of Δz, including contact effects, encoder error, and other state-dependent factors. The full objective is

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{inv}}\mathcal{L}_{\text{inv}} + \lambda_{\text{eq}}\mathcal{L}_{\text{equiv}}. \tag{3}$$

Why inverse dynamics is insufficient. A representation is action-decodable if some D satisfies $D(\Delta z(s, a)) = a$. For an action a, define its within-action variation as

$$V(a) = \operatorname{tr} \operatorname{Cov}_s[\Delta z(s, a)]. \tag{4}$$

Proposition 1 (Perfect action decoding does not imply concentration)**.** *If $\Delta z(s, a) = \rho(a)$ for every state s and ρ is injective, then the action can be decoded exactly from Δz, and $V(a) = 0$ for every action. The converse is false: for any $c > 0$, there exists a representation from which every action can be decoded exactly, while $V(a) \geq c$ for every action.*

(Proof in Appendix F.) The construction mirrors manipulation: one block of the latent follows the action, like the end effector, while another moves as a function of state, like a pushed object. A decoder can read the action from the first block and ignore the second, so decoding separates actions without making repetitions of one action share a reusable latent transition.

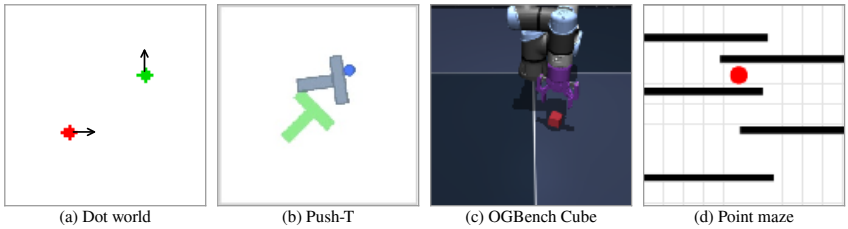
Learning shared action prototypes. For a fixed encoder and an unrestricted state-blind head,

$$\min_\rho \mathbb{E}\|\Delta z - \rho(a)\|_2^2 = \mathbb{E}_a \operatorname{tr} \operatorname{Cov}(\Delta z \mid a), \qquad \rho^*(a) = \mathbb{E}[\Delta z \mid a]. \tag{5}$$

The head absorbs the conditional mean but cannot explain the variance around it; reducing that variance requires the trainable encoder to reshape transition geometry. If the effect head also receives the current state, it can model state-specific changes directly, so the encoder is no longer encouraged to concentrate executions of one action. The equivariance loss alone admits the constant representation. The inverse loss prevents this collapse, and the predictive loss retains state-dependent outcomes, such as a push that moves the block only when the agent touches it.

3.3 Measuring concentration

Concentration means that knowing the action should explain most of the latent transition. Controlled-replay η^2 measures how the latent transition of a fixed action varies across starting states. Held-out action fit R^2 instead measures how well latent transitions can be predicted from actions in logged trajectories. All of our simulated environments support controlled replay; we additionally report action fit because it does not require reset access and can be applied to ordinary offline data.



(a) Dot world (b) Push-T (c) OGBench Cube (d) Point maze

Figure 3: **Environments.** (a) Dot world with two controlled dots; arrows show one primitive action each. (b)–(d) Pixel-based tasks: contact-rich pushing, sustained grasping, and maze navigation.

Controlled-replay concentration. We execute each action from multiple starting states and decompose the total covariance of the resulting latent transitions as

$$T = \mathrm{Cov}(\Delta z), \qquad W = \mathbb{E}_a \,\mathrm{Cov}(\Delta z \mid a), \qquad B = \mathrm{Cov}_a(\mathbb{E}[\Delta z \mid a]), \tag{6}$$

where $T = W + B$; W measures how much the latent transition of one action varies across states, and B how much the mean latent transitions of different actions vary. The controlled-replay concentration score is

$$\eta^2_{\mathrm{raw}} = 1 - \frac{\mathrm{tr}\, W}{\mathrm{tr}\, T} = \frac{\mathrm{tr}\, B}{\mathrm{tr}\, T}. \tag{7}$$

A score near one means that repeated executions of the same action produce similar latent transitions; a lower score means that the transition depends more on the starting state. Because the score uses Euclidean variance, it describes concentration under the learned metric.

Held-out action fit. Controlled replay requires access to the simulator state. To obtain a measure that can also be computed from logged trajectories, we fit a capacity-matched regressor $\hat{\rho}$ to predict latent transitions from actions. We train the regressor on one set of episodes and evaluate it on held-out episodes:

$$R^2(a \mapsto \Delta z) = 1 - \frac{\sum_{i \in \mathrm{test}} \|\Delta z_i - \hat{\rho}(a_i)\|_2^2}{\sum_{i \in \mathrm{test}} \|\Delta z_i - \overline{\Delta z}_{\mathrm{test}}\|_2^2}. \tag{8}$$

We call this held-out R^2 *action fit*. A high action fit means that the action alone predicts most of the latent transition on unseen trajectories. Unlike the controlled-replay score, action fit is an observational proxy and can be influenced by correlations between states and actions in the dataset. It can also be negative when the regressor performs worse than predicting the mean latent transition.

4 Experiments

4.1 Experimental setup

Environments. We use four simulated environments (Figure 3). The *dot world* is a 64×64 arena with one or two independently controlled dots. Each action is a pixel displacement, so the physical outcome of every action and of every composition is known exactly. We use a CNN encoder with d=64 and controlled replays of 24–48 actions from 400 states each. The three pixel-based tasks use the LeWM stack (Maes et al., 2026b) with a ViT-Tiny encoder (Dosovitskiy et al., 2021) (d=192) and the same hyperparameters without retuning. *Push-T* (Chi et al., 2023) tests contact-rich pushing with frame-skip-5 action chunks. *OGBench cube-single* (Park et al., 2025) tests sustained grasping; we measure concentration on sustained-grasp transitions and planning success on held-out tasks. *Maze*, a point-maze navigation task from stable-worldmodel (Maes et al., 2026a), has twenty training and five held-out corridor layouts; we measure concentration on free and colliding motions.

Matched training and baselines. The base model has the same encoder, predictor, inverse head, optimization budget, and replay data as EquiJEPA, differing only in λ_{eq}=0. Unless stated otherwise, results use three paired model seeds. We also compare a LeWM-style model, which replaces inverse dynamics with the LeWM isotropy guard, and DINO-WM-style frozen DINOv2 features (Oquab et al., 2024). The pixel-based tasks use λ_{inv}=30 and λ_{eq}=10, selected on Push-T by a disjoint sweep; the dot world uses λ_{inv}=λ_{eq}=10. Evaluation splits and protocols are in Appendix A.

Table 1: **Transition concentration.** Action fit is held-out R^2 of a linear regressor in the dot world and of a capacity-matched MLP on the pixel-based tasks. Replay η^2_{raw} is measured under each representation's learned metric and split by interaction regime. A dash marks an evaluation not run: the LeWM-style and DINO-WM-style baselines were not trained in the dot world. Entries are mean $\pm$ SD over three paired model seeds; DINO-WM-style features are frozen and deterministic. Bold marks the best mean. Worst-action statistics and diagnostics: Appendix Tables 4 and 5.

environment	metric	LeWM-style	DINO-WM-style	base (SMWM)	EquiJEPA
Dot world, one dot	action fit R^2 ↑	–	–	0.821 ±0.013	**0.9998 ±0.0001**
	replay η^2_{raw} ↑	–	–	0.836 ±0.010	**0.9997 ±0.0001**
Dot world, two dots	action fit R^2 ↑	–	–	0.888 ±0.018	**0.9989 ±0.0005**
	replay η^2_{raw} ↑	–	–	0.895 ±0.014	**0.9987 ±0.0007**
Push-T	action fit R^2 ↑	0.019 ±0.003	−0.061	0.688 ±0.014	**0.869 ±0.002**
	replay η^2_{raw} ↑, free	0.104 ±0.003	0.112	0.747 ±0.015	**0.910 ±0.001**
	replay η^2_{raw} ↑, contact	0.043 ±0.001	0.070	0.605 ±0.022	**0.833 ±0.003**
Cube	action fit R^2 ↑	0.100 ±0.007	0.073	0.825 ±0.009	**0.947 ±0.003**
	replay η^2_{raw} ↑, sustained grasp	0.033 ±0.0004	0.050	0.570 ±0.026	**0.872 ±0.005**
Maze	action fit R^2 ↑	−0.052 ±0.032	0.010	0.326 ±0.053	**0.687 ±0.028**
	replay η^2_{raw} ↑, free	0.061 ±0.018	0.052	0.481 ±0.047	**0.777 ±0.049**
	replay η^2_{raw} ↑, collision	0.080 ±0.028	0.057	0.206 ±0.014	**0.394 ±0.027**

4.2 Transition concentration

In the dot world, EquiJEPA raises η^2_{raw} above 0.998 (Table 1), and the worst-action score in the two-dot world rises from 0.453 to 0.992 (Table 4), so no subset of actions drives the average.

Across the three pixel-based tasks, EquiJEPA achieves the highest action fit and improves over the matched base for all three paired seeds in every environment.

The LeWM-style and DINO-WM-style baselines stay near zero under intervention in every pixel-based regime (0.033–0.112) although they retain task state (Table 2). The base model is trained to recover the action from consecutive latents, yet its controlled-replay concentration stays well below that of EquiJEPA, for example 0.747 versus 0.910 on contact-free Push-T and 0.570 versus 0.872 on Cube. This is the separation of Proposition 1: actions are decodable, but the latent transition of one action still varies with the state. Because replay intervenes on the starting state, EquiJEPA's gains are not explained by correlations in logged data. On both Push-T and Maze, contact or collision lowers concentration because an action's latent transition depends more on the current state.

4.3 Probing task-relevant state information

Concentrating latent transitions should not remove the state information needed by the world model. We therefore fit linear and MLP probes to frozen latent states (Table 2). Across decodable agent, object, and gripper quantities, every probe R^2 of EquiJEPA stays within 0.04 of the matched base, so concentration does not remove the probed state variables.

4.4 Rendering action prototypes in pixel space

To make the learned geometry visible in pixel space, we train a decoder on each frozen representation. The two decoders share architecture, data, and optimization, and both reach PSNR $\approx$31 dB. Every example in Figure 4 is held out from decoder training, so the decoder is a measurement instrument rather than part of EquiJEPA (protocol in Appendix C.4). On contact-free transitions, decoding $z_t + \rho(a_t)$ moves the agent to its position in the next observation. Under contact, it moves the hand but not the block, whose state-dependent motion belongs to the interaction residual. Over all $1{,}024$ held-out transitions, the rendered agent lands a median 2.1 px from its position in o_{t+1} under EquiJEPA against 9.5 px under the base on contact-free transitions, and 1.9 against 6.5 px under contact; the base lands farther from it in 96% of transitions.

Table 2: **Task-relevant information retained in frozen latent states.** Held-out R^2 of linear and MLP probes trained on frozen z_t and evaluated on disjoint episodes. Position rows average their coordinate dimensions. Push-T base and EquiJEPA use the 10-epoch full protocol; LeWM-style uses its 3-epoch budget and DINO-WM-style is frozen. Cube and Maze models use matched 3-epoch and 9-epoch budgets, respectively. Cube block yaw is omitted because no representation decodes it (all $R^2 \leq 0$). Appendix A provides the full probing protocol.

		linear probe R^2 ↑				MLP probe R^2 ↑			
environment	quantity	LeWM	DINO	base	EquiJEPA	LeWM	DINO	base	EquiJEPA
Push-T	agent position	0.940	0.961	0.992	0.989	0.964	0.945	0.966	0.977
	block position	0.933	0.962	0.939	0.925	0.939	0.926	0.950	0.945
	block orientation	0.591	0.758	0.508	0.468	0.754	0.481	0.665	0.637
Cube	block position	0.953	0.978	0.993	0.994	0.983	0.659	0.996	0.996
	gripper position	0.989	0.996	0.999	0.998	0.992	0.980	0.997	0.997
	gripper opening	0.872	0.850	0.962	0.960	0.939	0.789	0.983	0.985
Maze	agent position	−0.41	0.994	0.892	0.882	0.282	0.981	0.960	0.954

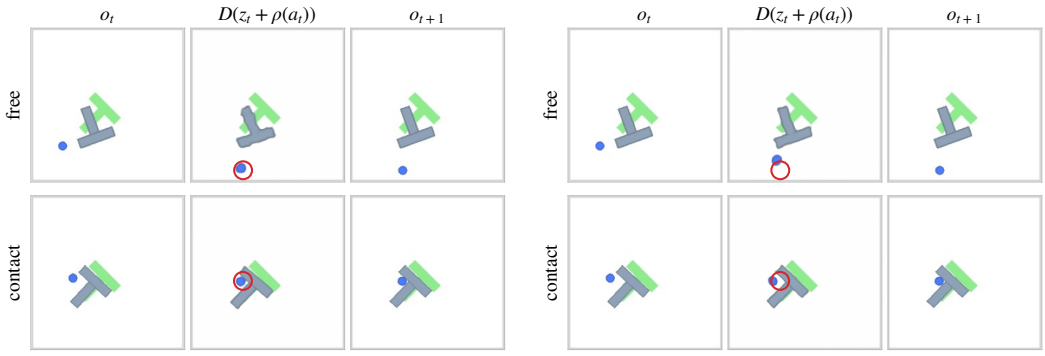


Figure 4: **Action prototypes rendered in pixels.** Left: EquiJEPA with its state-blind action prototype. Right: the base latent with a matched post-hoc $\hat{\rho}$. The red ring marks the agent in o_{t+1}. Each row is the held-out transition, contact-free (top) or contact (bottom), at the 75th percentile of the per-transition rendered-agent error gap between the two models.

4.5 Ablations

The ablations are single Push-T runs (Appendix Table 6). They identify two necessary components. First, the effect head must remain state-blind (Section 3.2): when it also receives the state, its head fit rises from 0.878 to 0.916, but the encoder's action fit returns to the base value. Second, the inverse loss prevents collapse: without it, the latent variance reaches exactly zero within one epoch. Replacing it with the LeWM isotropy guard avoids collapse but lowers action fit from 0.87 to 0.57. The inverse head should also see both latents: decoding the action from Δz alone leaves the head fit unchanged ($0.809 \rightarrow 0.837$) but lowers the linear block-angle probe from 0.468 to 0.153, so the pair input (z_t, z_{t+1}) keeps more state information.

5 Composition and Reuse of Action Structure

All evaluations here freeze the pretrained encoder; Appendices D and E give further controls.

5.1 Composing and Executing Action Prototypes

Composition in latent space. In the two-dot world, every training transition moves one dot along one signed axis, so there are eight physical primitives. Over eight consecutive steps in which both dots move, adding the single-dot prototypes of each step predicts the total latent transition with

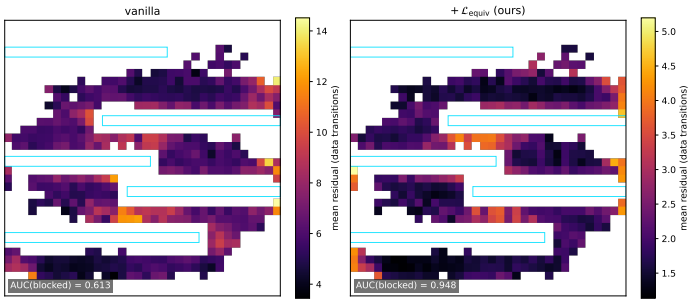


Figure 5: **Interaction residuals reveal walls in a held-out maze.** Colors show the mean residual in each cell; cyan outlines the walls. The displayed checkpoints have AUC 0.613 (base) and 0.948 (EquiJEPA); three-seed results are reported in the text. Color scales differ between panels.

relative error 0.097, against 0.551 for the base with a post-hoc $\hat{\rho}$; using the prototype of each joint action gives 0.190 against 0.709 (three model seeds).

Executing composed and transferred prototypes. In the two-dot world, the inverse head was trained only on actions that move one dot. Decoding the sum of two prototypes $\rho(a_1) + \rho(a_2)$ into a single action that moves both dots places each dot within 2 px of its target in 40.1% of trials with EquiJEPA, compared with 11.5% for the base (three model seeds, paired t=4.94). On Push-T, we add a prototype at a new starting state, decode the pair with the inverse head (Figure 2b), and execute the chunk in the simulator on 361 pairs, 214 of which make contact at the new state. The agent ends 2.67 versus 4.02 px from where the original action moves it (base with a ridge $\hat{\rho}$; paired t=10.0), 2.93 versus 4.33 px on contact-free pairs and 2.49 versus 3.80 px on contact pairs, where the pushed block also ends closer to its reference pose (2.25 versus 3.02 px). As an MPPI initialization, decoded goal displacements help low-budget planning for both models on Push-T and Cube but hurt it in Maze, where straight latent paths cross walls; with the same initialization, EquiJEPA is ahead on Cube and Maze and behind on Push-T (Appendix E).

Linear maps of the base latents. We test whether an invertible linear map A applied to the frozen base latents reproduces these results. The execution results cannot change under such a map: the least-squares prototype of the mapped latents is $A\hat{\rho}$ (exactly for the ridge fit on Push-T, and at the optimum of the fitted head in the two-dot world), and the inverse head composed with A^{-1} decodes the same action. We also fit two maps on frozen base latents over three model seeds: one trained with $\mathcal{L}_{\text{equiv}}$ under unit determinant and condition number at most 30, and one that shrinks directions outside the controlled-state subspace by the variance ratio measured on EquiJEPA. Neither reproduces composition in the two-dot world: the h=8 error of Table 3 is 0.226 and 0.182, against 0.523 for the base and 0.022 for EquiJEPA. In Cube planning at K=8, EquiJEPA raises success from 35.3% to 46.7%, while the two maps reach 30.0% and 36.4%.

5.2 Detecting walls from interaction residuals

Does the state-dependent variation left in the residual carry interpretable, useful information? We test this through wall localization. In the maze environment, a wall can block an action that would otherwise move the agent. For each frozen representation, we fit $\hat{\rho}$ on free-space transitions only. On held-out maze layouts, the residual $\|\Delta z - \hat{\rho}(a)\|$ measures how far the observed change departs from that action's expected effect in open space. It separates blocked from free motion with AUC 0.943 ± 0.007 for EquiJEPA, versus 0.664 ± 0.087 for the base (mean $\pm$ SD across three model seeds). Figure 5 shows how these deviations reveal walls: the prototype captures the usual action effect, while the residual captures where the surroundings change it.

5.3 A sparse dictionary of latent transitions

We fit a top-k sparse autoencoder (Makhzani & Frey, 2014; Gao et al., 2025) to the latent transitions Δz_t of each frozen encoder. It assigns each latent transition nonnegative coefficients c on at most k

Table 3: **Dictionary primitives and composition in the two-dot world.** All columns use one sparse dictionary protocol per frozen representation, over three model seeds. FVU is the fraction of latent-transition variance not reconstructed by the dictionary. "Atom ID" is top-1 primitive-identification accuracy up to a permutation of atom labels. Held-out error reconstructs unseen within-step combinations, h=8 is the relative error of summed dictionary reconstructions over eight steps, and physical error is measured after decoding a composed goal into simulator pixels.

representation	FVU ↓	held-out err. ↓	h=8 err. ↓	physical px ↓	atom ID ↑
base	.0836	.2239	.5225	.4737	.949
EquiJEPA	**.0007**	**.0273**	**.0217**	**.1112**	**1.000**

of K unit-norm atoms, the rows of the dictionary V, and reconstructs $\widehat{\Delta z} = c^\top V + b$, where b is the mean latent transition (objective and optimization in Appendix C). K is fixed in advance: K=8 for the two-dot world and K=16 for Push-T and Cube. Because several atoms can be active at once, a diagonal latent transition is the sum of a horizontal and a vertical atom, whereas a single-code VQ codebook needs a separate codeword for each such combination.

Primitive recovery and composition. The two-dot dictionary is fitted on single-primitive transitions (Table 3). For EquiJEPA, its atoms match the eight primitives one-to-one: the atom with the largest coefficient identifies the executed primitive on every held-out single-dot transition (Atom ID 1.000). When both dots move in the same held-out transition, the sparse code activates both corresponding atoms in all $10{,}000$ cases. Summing the dictionary reconstructions of eight consecutive transitions predicts the total latent transition with relative error 0.0217, against 0.5225 for the base.

Structure on embodied tasks. On Push-T, decoding a fixed atom through the inverse head at 96 held-out states produces consistent action directions (median cosine 0.89 to the atom's mean readout). With K=32 atoms and top-4 codes, the Push-T dictionary leaves an FVU of 0.091 on EquiJEPA's latent transitions against 0.174 on the base's. On Cube, the dictionary reaches an FVU of 0.047, while the same-size VQ codebook has $4.3\times$ larger error. When decoded through the inverse head, the learned Cube atoms produce distinct five-step action patterns: some raise or lower the end effector, some open or close the gripper, and others move the end effector while rotating the wrist. The atoms are specific to each task (Appendix Figures 10 and 11).

5.4 Reusing a frozen action dictionary with a new command interface

We ask whether a frozen dictionary learned with one command interface can be used through another. On the same recorded Push-T transitions, interface A writes each action chunk as five 2-D commands (10 values); interface B halves the command scale, rotates the axes by $45°$, and uses three 2-D commands per chunk (6 values). We freeze the encoder and the dictionary V learned with interface A and train only an interface-B effect head $\rho_B(a_B) \approx z_{t+1} - z_t$ and inverse head $g_B(z_t, z_{t+1}) \approx a_B$. For paired commands that describe the same transition, ρ_A and ρ_B give prototypes with cosine similarity 0.974. Decoding a frozen atom v_k through either interface, $g_A(z, z+v_k)$ or $g_B(z, z + v_k)$, gives nearly the same physical direction in world coordinates (cosine 0.995), and the success of a greedy controller that follows the dictionary decomposition is nearly unchanged (58.7% for B versus 58.0% for A). Only the command-specific heads are refitted.

6 Conclusion

EquiJEPA goes beyond action decodability by encouraging the same action to produce consistent latent transitions across states. Across four simulated environments, its state-blind effect head increases transition concentration while keeping agent and object state decodable. The resulting prototypes support composition, more accurate execution at new states and of unseen action combinations, and reuse through a new command interface. Interaction residuals complement these shared prototypes by revealing where the environment changes an action's effect, as demonstrated by wall localization in Maze. Together, these results support organizing latent world models around reusable action effects and their state-dependent deviations.

AI USE STATEMENT

We used generative AI tools to edit the paper for readability, draft parts of the text, search for and verify related literature, and format references. We have reviewed all AI-assisted work: every reported number was traced to a per-run result file, generated code was run and its outputs checked by the authors, and each added citation was checked. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

REPRODUCIBILITY STATEMENT

All datasets are public, and every result traces to a per-run JSON artifact. We release training code, controlled-replay collectors, dictionary and interaction-residual analyses, and planning evaluations. A world-model run uses one consumer GPU for two to four hours; cached diagnostics take minutes.

REFERENCES

Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

Stefan Banach. Sur l'équation fonctionnelle $f(x+y) = f(x) + f(y)$. *Fundamenta Mathematicae*, 1:123–124, 1920.

Daniel A. Braun, Ad Aertsen, Daniel M. Wolpert, and Carsten Mehring. Motor task variation induces structural learning. *Current Biology*, 19(4):352–357, 2009.

Jake Bruce, Michael D. Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, et al. Genie: Generative interactive environments. In *International Conference on Machine Learning*, 2024.

Hugo Caselles-Dupré, Michael Garcia Ortiz, and David Filliat. Symmetry-based disentangled representation learning requires interaction with environments. In *Advances in Neural Information Processing Systems*, 2019.

Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.

Zichen Jeff Cui, Hengkai Pan, Aadhithya Iyer, Siddhant Haldar, and Lerrel Pinto. DynaMo: In-domain dynamics pretraining for visuo-motor control. In *Advances in Neural Information Processing Systems*, 2024.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations*, 2024.

Opher Donchin, Joseph T. Francis, and Reza Shadmehr. Quantifying generalization from trial-by-trial behavior of adaptive systems that learn with basis functions: Theory and experiments in human motor control. *The Journal of Neuroscience*, 23(27):9032–9045, 2003.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. In *International Conference on Learning Representations*, 2025.

Hafez Ghaemi, Eilif B. Muller, and Shahab Bakhtiari. seq-JEPA: Autoregressive predictive learning of invariant-equivariant world models. *arXiv preprint arXiv:2505.03176*, 2025.

Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024.

Irina Higgins, David Amos, David Pfau, Sebastien Racaniere, Loic Matthey, Danilo Rezende, and Alexander Lerchner. Towards a definition of disentangled representations. *arXiv preprint arXiv:1812.02230*, 2018.

Petr Ivashkov, Randall Balestriero, and Bernhard Schölkopf. Sensorimotor world models: Perception for action via inverse dynamics. *arXiv preprint arXiv:2606.20104*, 2026.

Yuxin Jiang, Yuchao Gu, Ivor W. Tsang, and Mike Zheng Shou. Olaf-World: Orienting latent actions for video world modeling. In *International Conference on Machine Learning*, 2026.

Hamza Keurti, Hsiao-Ru Pan, Michel Besserve, Benjamin F. Grewe, and Bernhard Schölkopf. Homomorphism autoencoder: Learning group structured representations from observed transitions. In *International Conference on Machine Learning*, 2023.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

Thomas Kipf, Elise van der Pol, and Max Welling. Contrastive learning of structured world models. In *International Conference on Learning Representations*, 2020.

Yann LeCun. A path towards autonomous machine intelligence. OpenReview, 2022. URL `https://openreview.net/pdf?id=BZ5a1r-kVsf`.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, et al. stable-worldmodel: A platform for reproducible world modeling research and evaluation. *arXiv preprint arXiv:2605.21800*, 2026a.

Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026b.

Alireza Makhzani and Brendan Frey. k-sparse autoencoders. In *International Conference on Learning Representations*, 2014.

Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, 1996.

Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.

Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *International Conference on Learning Representations*, 2025.

Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.

Dominik Schmidt and Minqi Jiang. Learning to act without actions. In *International Conference on Learning Representations*, 2024.

Wacław Sierpiński. Sur l'équation fonctionnelle $f(x+y) = f(x) + f(y)$. *Fundamenta Mathematicae*, 1:116–122, 1920.

Uladzislau Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems*, 2025.

Junhan Sun, Hao Zhao, and Guofeng Zhang. INTACT: Isomorphic intent-to-action learning for search-free world models. *arXiv preprint arXiv:2607.26056*, 2026.

Kurt A. Thoroughman and Reza Shadmehr. Learning of action through adaptive combination of motor primitives. *Nature*, 407(6805):742–747, 2000.

Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017.

Elise van der Pol, Daniel Worrall, Herke van Hoof, Frans Oliehoek, and Max Welling. MDP homomorphic networks: Group symmetries in reinforcement learning. In *Advances in Neural Information Processing Systems*, 2020.

Ying Wang, Oumayma Bounou, Yann LeCun, and Mengye Ren. AdaJEPA: An adaptive latent world model. *arXiv preprint arXiv:2606.32026*, 2026a.

Yuhai Wang, Jiawei Xia, Rongxuan Zhou, Xiao Hu, Yongliang Shi, Jing Du, and Yang Ye. PRISM: Prior-guided imagination sampling in world models. *arXiv preprint arXiv:2606.07974*, 2026b.

Manuel Watter, Jost Tobias Springenberg, Joschka Boedecker, and Martin Riedmiller. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advances in Neural Information Processing Systems*, 2015.

Grady Williams, Paul Drews, Brian Goldfain, James M. Rehg, and Evangelos A. Theodorou. Aggressive driving with model predictive path integral control. In *IEEE International Conference on Robotics and Automation*, 2016.

Seonghyeon Ye, Joel Jang, Byeongguk Jeon, Sejune Joo, Jianwei Yang, Baolin Peng, Ajay Mandlekar, Reuben Tan, Yu-Wei Chao, Bill Yuchen Lin, Lars Liden, Kimin Lee, Jianfeng Gao, Luke Zettlemoyer, Dieter Fox, and Minjoon Seo. Latent action pretraining from videos. In *International Conference on Learning Representations*, 2025.

Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning*, 2025.

A EXPERIMENTAL SETUP AND EVALUATION PROTOCOLS

This appendix collects the evaluation protocols used throughout the paper. Environment definitions and matched training budgets are summarized in Section 4.1; controlled-replay scoring is defined in Section 3.3. Dictionary, decoder, transfer, and planning protocols accompany their results in Appendices C, D, and E.

A.1 STATE PROBES AND EVALUATION SPLITS

Table 2 follows the probe format of Ivashkov et al. (2026). Probes are fit and evaluated on disjoint episodes, a stricter protocol than a shuffled-transition split; under the latter, the same MLP probe reaches 0.999 on agent position. Absolute values are therefore not directly comparable across papers, but every model within each row uses the same split and probe. Position scores average their coordinate dimensions. For Push-T, the base and EquiJEPA use the 10-epoch full checkpoints, LeWM-style uses its 3-epoch checkpoint, and DINO-WM-style is frozen. Cube uses the 3-epoch training setup and Maze the 9-epoch setup, with added baselines trained for the corresponding budget. Cube yaw is not decoded by any representation; the $(\sin, \cos)$ and quaternion targets are equally poor, ruling out the angle encoding as the cause. On Maze, the LeWM-style representation also fails to retain agent position without the inverse-dynamics guard (Section 4.5).

A.2 PROTOCOL FOR FIGURE 1

The three arms differ only in their loss weights: EquiJEPA uses $\lambda_{\text{inv}}{=}30$ and $\lambda_{\text{eq}}{=}10$, SMWM uses $\lambda_{\text{inv}}{=}30$ alone, and the LeWM-style arm uses the SIGReg isotropy guard at 0.09 with no inverse-dynamics and no equivariance term. Every other setting, including the seed, matches the corresponding published arm of Table 1.

These are separate runs, trained to retain intermediate weights, because the published runs keep only their final checkpoint. Intermediate weights are saved at 16 logarithmically spaced optimizer steps up to 22862 (three epochs), and each one is measured on the controlled-replay bank of Section 3.3 (24 action chunks $\times$ 96 start states) using its pre-declared validity filter and contact labels. The bank is encoded once per checkpoint, so all arms and all checkpoints see byte-identical frames.

The score plotted is η^2_{raw} pooled over every valid rollout rather than split by interaction regime, which is why it cannot be read against Table 1 directly. Split by regime, the final checkpoints give 0.901 contact-free and 0.811 under contact for EquiJEPA against 0.759 and 0.618 for SMWM, and 0.100 and 0.040 for the LeWM-style arm; the published values are $0.910/0.833$, $0.747/0.605$ and $0.104/0.043$. Held-out action fit on the same checkpoints is 0.857, 0.677 and 0.027 against a published 0.869, 0.688 and 0.019.

The latent-transition panels use a two-dimensional basis fitted once on EquiJEPA's final checkpoint and then held fixed for every panel, so positions are comparable across the row; fitting a basis per panel would give each its own rotation. Colour encodes the net direction of the action chunk. Direction is cyclic, so a cool-to-warm ramp would normally introduce a discontinuity; the 24 chunk directions leave an 86° gap, and the seam is placed inside it.

Table 4: **Controlled-replay concentration and clean state decodability.** **(a)** Dot world: the concentration score η^2_{raw} is reported as mean (worst action). Every entry averages three paired model seeds. In the two-dot training data, each action moves only one dot; the final row tests unseen actions that move both dots simultaneously. The last column is a held-out linear action fit. **(b)** Push-T: each five-action chunk is replayed from 96 states. The three displacement columns are held-out $1 - \text{SSE}/\text{SST}$ estimates and "mean" is their macro-average; unlike the population trace ratio, an out-of-sample estimate may be negative. The last two columns are clean MLP state probes. All displacement scores use each model's own Euclidean metric.

(a) Dot world

model	one-dot world η^2_{raw}	two-dot world single-dot actions η^2_{raw}	linear action fit $R^2(a \mapsto \Delta z)$ (1-dot / 2-dot)
base	0.836 (0.546)	0.895 (0.453)	0.821 / 0.888
EquiJEPA ($+\mathcal{L}_{\text{equiv}}$)	**0.9997 (0.9977)**	**0.9987 (0.9915)**	**0.9998 / 0.9989**
EquiJEPA, both dots move (unseen combination)	—	**0.9989 (0.9987)**	—

(b) Push-T

		held-out replay R^2				state probe $R^2(z_t \mapsto y_t)$	
model / target	$R^2(a \mapsto \cdot)$ target in row	small	mid	large	mean	block pos.	block angle
base	0.595	0.866	0.694	−0.176	0.461	0.950	0.665
λ_{eq}=10	**0.795**	**0.913**	**0.838**	**0.206**	**0.653**	0.945	0.637
ground-truth state displacement Δs	0.554	0.919	0.571	−0.119	0.457	—	—

B Additional representation results and ablations

B.1 Controlled-replay concentration and state information

The tables below separate transition concentration from state decodability, and report the training budgets and evaluation regimes for each comparison.

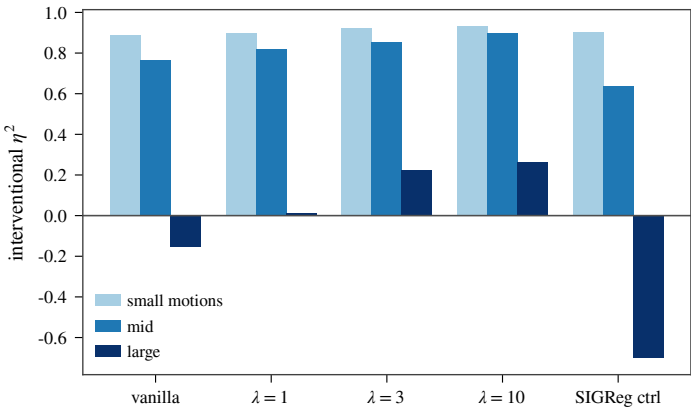


Figure 6: **Held-out replay R^2 by displacement tercile** (companion to Table 4, with the LeWM-style control added). EquiJEPA has the highest learned-metric score at every scale; the gain concentrates on large motions, where the fitted base prototype $\hat{\rho}$ generalizes worse than the held-out grand mean and the resulting R^2 is negative.

Comparison with other representation constraints. Tables 1 and 2 evaluate two properties separately: whether latent transitions are predictable from actions, and whether the latent states retain

Table 5: **Action fit and state information on Push-T** (the dual gate). "Action fit" is the post-hoc, capacity-matched MLP $R^2(a \mapsto \Delta z)$. State probes are linear/MLP $R^2(z_t \mapsto y_t)$ from the frozen z_t, with y_t given by the column label. Each row is a single run, so the 3-epoch entries (0.674, 0.869) are the seed-0 members of the three-seed means reported in Table 1.

model	budget	$R^2(a \mapsto \Delta z)$	block$_x$	block$_y$	block$_{\text{ang}}$
base	3 ep	0.674	.931/.923	.940/.933	.467/.453
base	10 ep	0.595 $\downarrow$	.932/.945	.947/.956	.508/.665
λ_{eq}=10	3 ep	0.869	.917/.929	.914/.916	.376/.404
λ_{eq}=10	10 ep	**0.795**	.912/.933	.939/.957	.468/.637
λ_{eq}=30	3 ep	0.898	.912/.920	.921/.922	.321/.379 $\downarrow$

Table 6: **Ablations of the state-blind objective and inverse-dynamics guard.** Each row modifies EquiJEPA. "Action fit" denotes $R^2(a \mapsto \Delta z)$; "head fit" is the held-out R^2 of the trained ρ. "blk-ang" is the linear block-angle probe. All entries are single runs. Rows 1 and 3 use the 3-epoch budget of Table 1; row 4 uses the 10-epoch budget of Table 5.

change	transition concentration	state decodability	interpretation
ρ conditioned on the state	head fit $0.878 \rightarrow 0.916$; encoder action fit returns to the base value	—	state-blind head is needed to constrain the encoder
inverse-dynamics guard removed	both remaining losses reach 10^{-8} within one epoch	latent variance exactly 0	$\mathcal{L}_{\text{equiv}}$ requires an anti-collapse guard
guard kind $\rightarrow$ isotropy (Maes et al., 2026b)	action fit $0.87 \rightarrow 0.57$	no collapse	isotropy preserves marginal variance but lowers action fit
guard input $\rightarrow$ $g_\omega(\Delta z)$, $\mathcal{L}_{\text{equiv}}$ kept	head fit unchanged ($0.809 \rightarrow 0.837$)	blk-ang $0.468 \rightarrow 0.153$; largest-motion concentration score: $0.206 \rightarrow -0.001$	pair input preserves more state information

task-relevant state information. The LeWM-style and DINO-WM-style representations retain state information in most settings, but their action fits are at most 0.10. On Push-T, for example, the LeWM-style model has the best block-angle probe but an action fit of only 0.02. On the maze, its agent-position probe falls to 0.28 without inverse-dynamics training; the inverse-trained models score between 0.96 and 0.99. Across all three environments, EquiJEPA improves the action fit while matching the state probes of its corresponding base model.

B.2 Objective ablations and loss-weight sensitivity

Table 7: **Held-out replay R^2: the λ_{eq} sweep** (3-epoch protocol of Table 4; monotone at every displacement scale).

model	small	mid	large	mean
base	0.887	0.763	-0.152	0.499
λ_{eq}=1	0.898	0.817	0.011	0.575
λ_{eq}=3	0.924	0.855	0.226	0.668
λ_{eq}=10	**0.932**	**0.897**	**0.264**	**0.697**

Additional checks. A linear ρ reaches an action fit of 0.92 and gives the lowest multi-step composition error, but also the weakest block-angle probe. Huber and Welsch penalties do not help because contact occurs in 45% of the expert frames. Adding a multi-step consistency loss gives no further improvement, consistent with Proposition 3. Finally, the interaction residual detects contact without labels (AUC 0.73). As a sanity check, an effect head trained with λ_{eq}=0 has a held-out head fit of $R^2 \approx 0$.

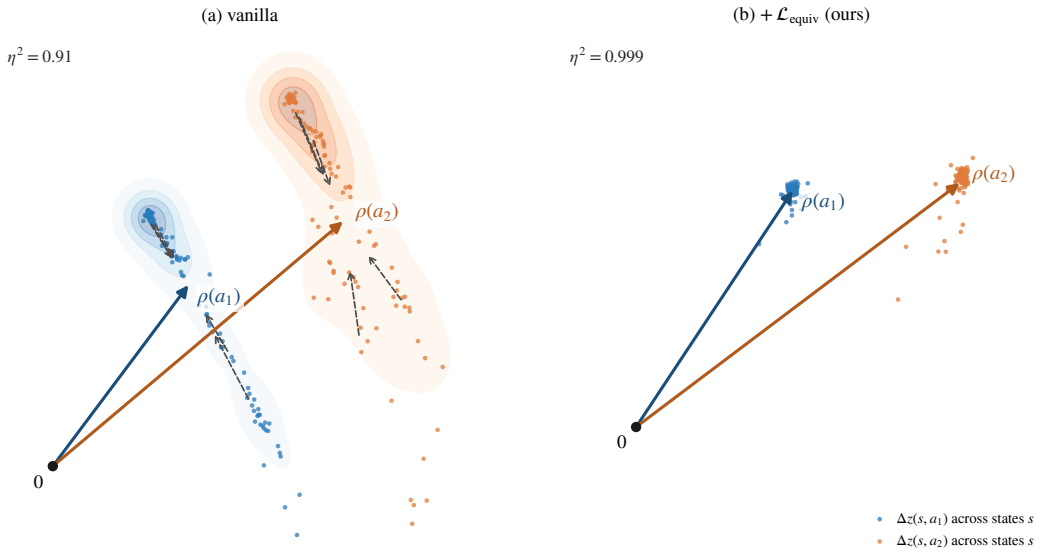


Figure 7: **The transition-concentration pressure**, drawn in the model's own latent-transition space (real probe data, two-dot world; per-model PCA, panels on one scale). Two actions are each replayed from 400 states; every dot is one $\Delta z(s, a_i)$. **(a)** Without the constraint, state-dependence smears each action into a cloud. The blind head contributes one vector per action ($\rho(a_i)$, bold arrow, the cloud's mean), so $\mathcal{L}_{\text{equiv}}$ decomposes into a mean term the head removes plus a variance term only the *encoder* can remove: blindness routes the pressure into the encoder. **(b)** After training, the 400 states of every action collapse onto their action prototype (η^2_{raw}=0.999).

Algorithm 1 Top-k SAE with dead-atom revival, best of R restarts.

Require: latent transitions $\{\Delta z_i\}$, size K, sparsity k
1: **for** init $\in \{k$-means++, random$\}$ and seed $\in \{1, 2, 3\}$ **do**
2: $b \leftarrow \mathrm{mean}(\Delta z)$; initialize unit-norm atoms V (and $W_e \leftarrow V$ for the k-means initialization)
3: **for** 60 epochs over minibatches x **do**
4: $c \leftarrow \mathrm{TopK}_k(\mathrm{ReLU}(W_e(x - b) + b_e))$
5: Adam step on $\|x - (c^\top V + b)\|^2$; renormalize rows of V
6: every 10 epochs: reassign every *dead* atom (activation rate below $1/(50K)$) to the direction of a currently worst-reconstructed sample
7: **end for**
8: **end for**
9: **return** the restart with the lowest held-out FVU

Table 8: Each constraint of the dictionary learner carries one claim of the paper.

design choice	mechanism	claim it carries
nonnegative codes (ReLU)	no cancellation between atoms	separate signed transition directions; codes are instructions
unit-norm atoms	magnitude lives in the coefficients	coefficients as amounts; addition
hard top-k	few atoms explain each Δz	sparsity; identifiability of the frame
dead-atom revival	all K atoms participate	no wasted capacity (usage $16/16$)
best of 6 restarts	escapes winner-take-all local optima	stable frame recovery (ablation d)

C Action dictionaries and visualizations

C.1 Dictionary learning and implementation

The dictionary learner is a self-contained top-k sparse autoencoder of roughly one hundred lines, written for this paper rather than taken from a package. The design follows the sparse-autoencoder literature (Cunningham et al., 2024), and every constraint it adds carries one claim of the paper, as summarized in Table 8.

Architecture. A linear encoder maps a centered latent transition to K scores, a ReLU makes the codes nonnegative, and a hard top-k mask keeps only the k largest coefficients:

$$c \;=\; \mathrm{TopK}_k\big(\mathrm{ReLU}(W_e(\Delta z - b) + b_e)\big), \qquad \widehat{\Delta z} \;=\; c^\top V + b,$$

where the rows of V are the atoms, re-projected to unit norm after every optimizer step, and b is initialized to the data mean. Training minimizes the reconstruction error $\|\Delta z - \widehat{\Delta z}\|^2$ with Adam (Kingma & Ba, 2015); there is no other loss term.

Hyperparameters and cost. $K \in \{8, 16, 32\}$ with $k \in \{2, 4\}$; 60 epochs at batch size 8192 and learning rate 10^{-3}; six restarts (two initializations $\times$ three seeds) selected by held-out FVU; revival threshold $1/(50K)$, checked every ten epochs. On 150,000 cached latent transitions one restart trains in one to two minutes on a single GPU, so the full protocol costs roughly ten minutes; the same code (with the same settings) produces the toy, Push-T, Cube, axis-only, and base-model dictionaries.

VQ baseline. The vector-quantization baseline is k-means++ with empty-cluster re-seeding, run to convergence on the same data; reconstruction assigns each latent transition to its nearest centroid. It shares the data, the size K, and the evaluation with the SAE, so the comparison isolates the coding scheme — additive versus holistic — rather than the training pipeline.

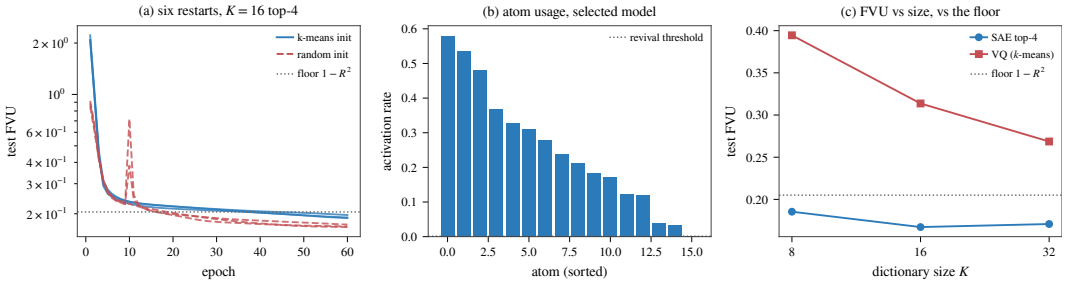


Figure 8: **Dictionary training in practice** (Push-T, EquiJEPA latent transitions, K=16 top-4). **(a)** Across six independent training runs, final held-out FVU ranges from 0.167 to 0.197, showing that dictionary quality depends on initialization; we therefore select the run with the lowest held-out FVU. In one randomly initialized run, resetting an unused atom at epoch 10 temporarily increases the error, after which continued training reduces it. **(b)** In the selected $K = 16$ dictionary, every atom is activated on the data. Usage is uneven—some atoms are selected frequently and others rarely—but all activation rates remain above the threshold used to identify dead atoms. **(c)** Held-out reconstruction error for different dictionary sizes. At every K, the top-4 SAE reconstructs latent transitions more accurately than a same-size k-means codebook. The dotted line shows the variance left unexplained by an action-only regressor. It is included only as a reference: unlike that regressor, the SAE observes the full displacement and can also encode state-dependent variation.

C.2 Primitive recovery and action readouts

C.3 Dictionary structure on OGBench Cube

The identical training setup transfers to OGBench cube-single with no retuning. Under a sustained grasp the learned-metric action fit $R^2(a \mapsto \Delta z)$ rises from 0.825 to 0.947 with the clean block probes unchanged (Table 2). The dictionary FVU is 0.047, close to the learned-metric unexplained-variance fraction $1 - R^2(a \mapsto \Delta z) \approx 0.05$. VQ stays 4.3× above it. Figure 11 decodes each atom: under sustained grasp the dictionary allocates its atoms to gripper, lift, and yaw structure.

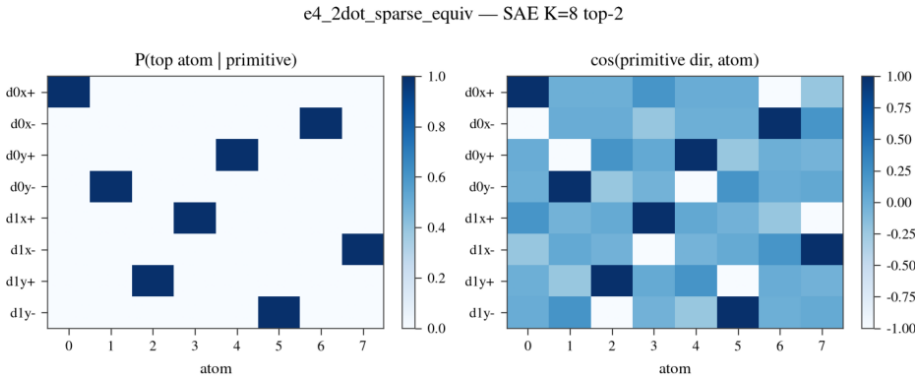
C.4 Pixel-decoder protocol and additional renders

Protocol. The decoder used in Figures 4 and 13, as well as the supplementary renders below, is a measurement instrument, not part of the method. Three rules keep it evidentially clean. *Post hoc*: it is trained after world-model training on frozen embeddings, so no reconstruction gradient ever shapes the representation — whatever it renders was already in the latent (the PLDM convention). *Held out*: the 25k cached transitions are split at the transition level; every rendered example comes from the decoder's eval split. *Matched instruments*: both models receive the identical architecture, data, and training settings, and converge to the same reconstruction quality (PSNR 30.9 versus 31.0 dB), so differences under latent pushes are properties of the latent geometry, not of the decoder.

Architecture and training. $z \in \mathbb{R}^{192} \to$ linear $\to$ $192{\times}14{\times}14$ $\to$ four stages of (nearest-neighbour upsample, two $3{\times}3$ convolutions, GroupNorm, SiLU) $\to 3{\times}224^2$ sigmoid; $\sim$8.5M parameters. Charbonnier loss with moving-pixel weighting ($w = 1 + 8\,|x - \bar{x}|$ against the mean frame; the agent disk covers $\sim$0.1% of pixels and plain MSE erases it), plus Gaussian noise on z at 0.05 of the per-dimension standard deviation during training, which keeps renders sharp on off-manifold inputs such as $z + \rho(a)$. AdamW (Loshchilov & Hutter, 2019), learning rate $3{\times}10^{-4}$, 16 epochs.

Latent edits rendered in pixels. For visualization, we train a decoder after freezing each representation. The two decoders use the same architecture, data, and optimization and reach similar reconstruction quality (PSNR $\approx$31 dB). All edited transitions are held out from decoder training (Appendix C.4). In Figure 13, we add the same action prototype $\rho(a^*)$ to six starting latents. EquiJEPA places the agent in the target ring in all six cases. Applying the best post-hoc direction to the

(a) top-k SAE



(b) VQ codebook

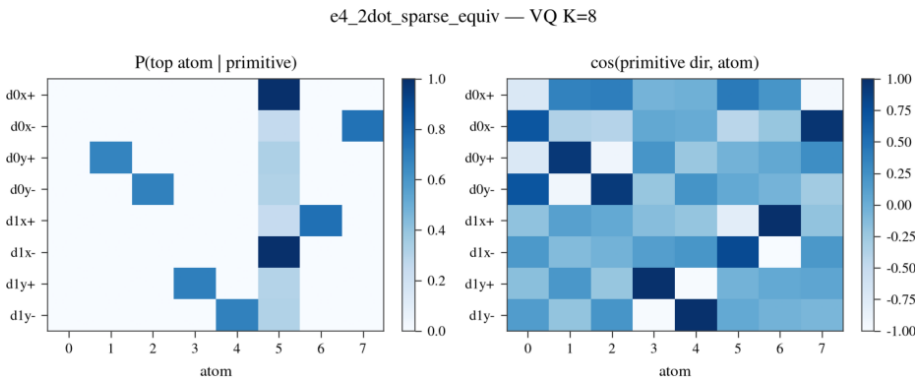


Figure 9: **Exact recovery versus holistic coding** (2-dot primitive-sparse world, $K=8$). Left of each panel: the probability that a transition of the row primitive is dominated by the column atom; right: the cosine between the ground-truth primitive direction and the atom vector. **(a)** The SAE assignment is a permutation with cosines of 1.00 on the matched cells and -1 at each primitive's signed opposite: the true signed generators are recovered exactly, up to renaming. **(b)** The VQ codebook of equal size has no permutation structure, since one holistic code per delta cannot represent signed, magnitude-carrying primitives.

base latent either leaves the agent in place, removes it from the reconstruction, or misses the ring. Figure 4 shows contact cases: the edit moves the hand while leaving the block at its original pose.

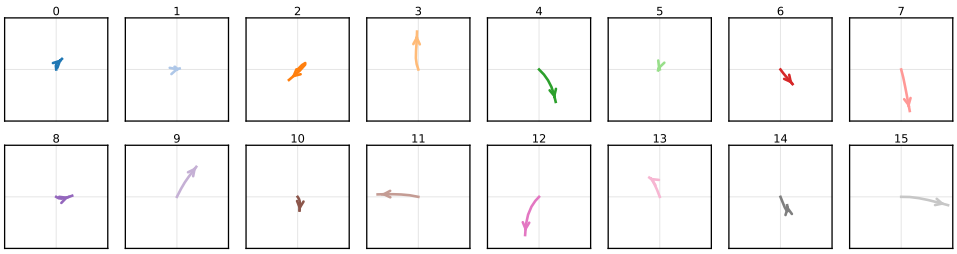


Figure 10: **What the atoms do.** Each of the 16 atoms of the Push-T dictionary, decoded by the planner's own readout: the trained inverse head applied to one unit of the atom, $\hat{a} = g_\omega(z,\, z + v_k)$, averaged over 96 held-out states (arrowhead marks the final step; standardized action units). The readouts are consistent across states (median cosine to the mean 0.89), so each stroke is the motion the planner executes for one unit of that atom; together the atoms tile the movement directions. Section D.1 shows that these identities persist across actuation interfaces.

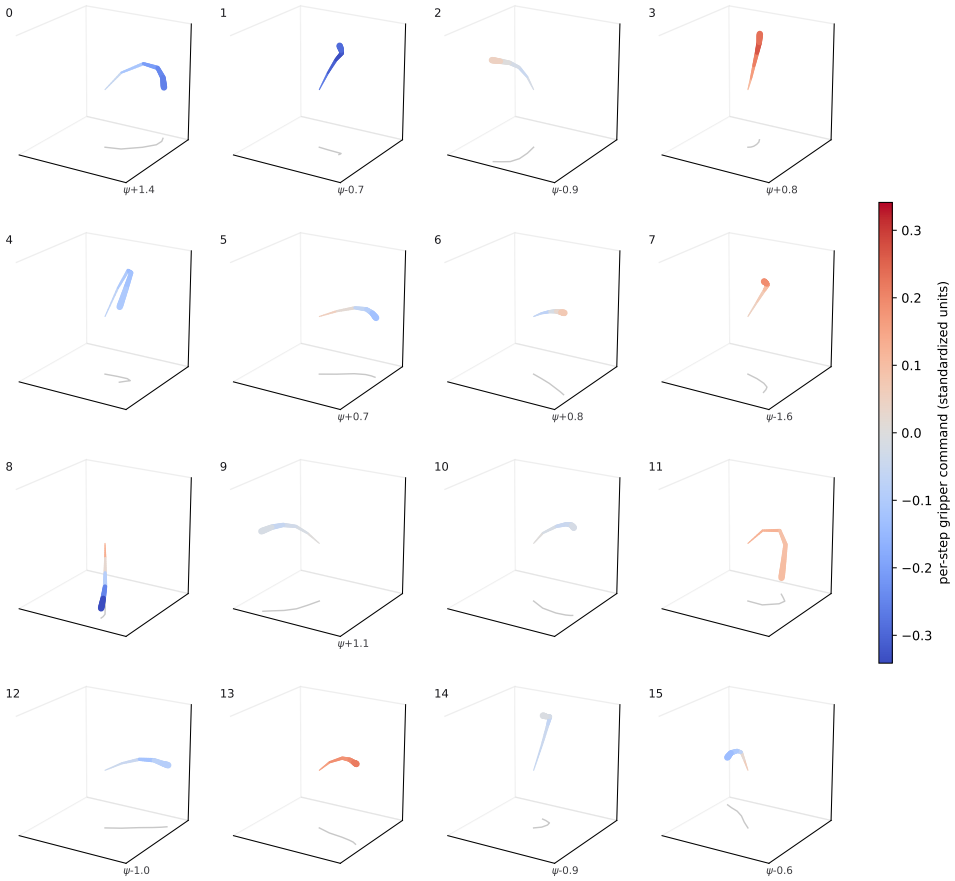


Figure 11: **What the Cube atoms do** (companion to Figure 10; same readout — each atom decoded by the trained inverse head, $\hat{a} = g_\omega(z,\ z{+}s_k v_k)$, averaged over 96 held-out states, with s_k the atom's median active coefficient; standardized action units; the stroke tapers from thin at the start to thick at the final step, so the width is the direction of time). The cube task is three-dimensional, so each atom is drawn as its xyz path (gray floor projection as a depth cue); the two remaining degrees of freedom are the segment color — the per-step gripper command — and the tag ψ, the atom's net five-step wrist-rotation command (shown when $|\psi| > 0.5$). Where the Push-T atoms tile planar directions, the Cube dictionary allocates its atoms to lift/lower arcs, gripper action (saturated red/blue segments), and mixed strokes with modest wrist rotation: the dictionary follows the environment's action space. Across states, the translation channels of the readout are consistent (median cosine 0.67 to the anchor mean); the wrist and gripper channels vary more (0.30) — those commands are read relative to the current wrist and gripper pose, so their state dependence is expected.

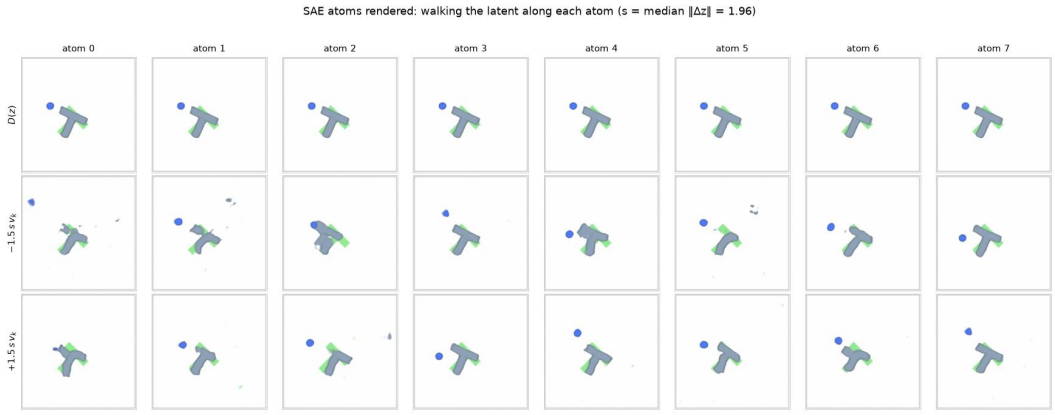


Figure 12: **Dictionary atoms, rendered.** Walking the latent along each SAE atom ($\pm 1.5\times$ the median $\|\Delta z\|$) and decoding: atoms act as signed hand-motion directions with antisymmetric pairs, and the block does not move — atoms live in the action-achievable subspace of Δz by construction.

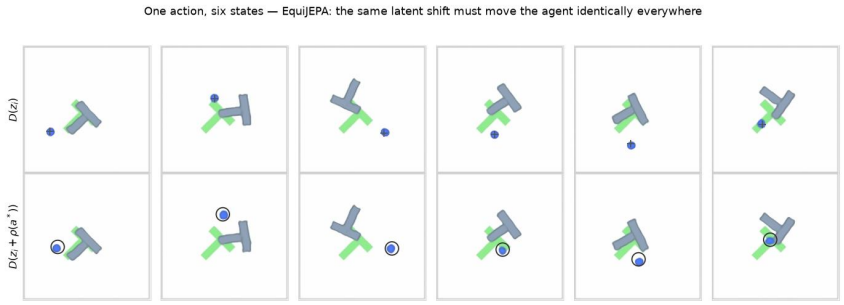


Figure 13: **One action, six states, rendered.** One action chunk a^* ($\approx$55 px) added in latent space at six held-out states and decoded; ring = ground-truth landing spot. $D(z_i + \rho(a^*))$ lands in the ring at every state; the base latent leaves the agent in place, deletes it, or misses at all six (Appendix Fig. 14).

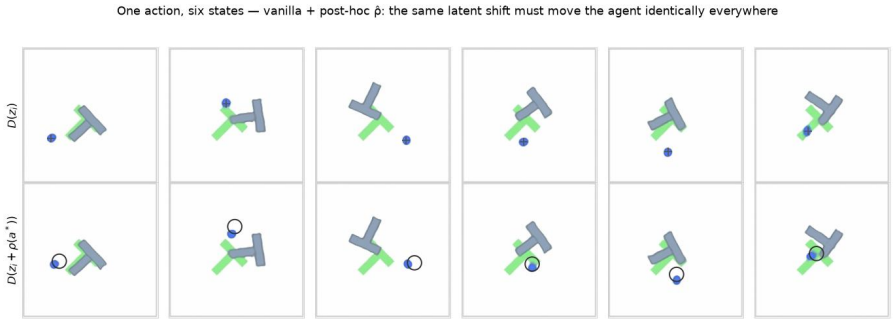


Figure 14: **One action, six states on the base latent** (companion to Figure 13; identical decoder quality and protocol). Pushed along its best post-hoc action direction, the base latent leaves the agent in place, deletes it, or misses the ring.

Table 9: **Transfer results.** Each row changes one component and keeps the rest fixed. The dictionary is frozen in rows 2–4; row 1 fits a projector on top of frozen DINOv2 features. "Action fit" denotes held-out $R^2(a \mapsto \Delta z)$. Additional results appear in this appendix (Tables 10, 11; Figures 16, 15).

axis	what changes (rest frozen)	result	scope
foundation backbone	encoder → frozen DINOv2 + two-layer projector	raw-feature action fit $R^2(a \mapsto \Delta z) \approx 0$; state probes 0.90–0.96; a projector with the loss reaches action fit 0.70 (0.38 without it)	below end-to-end 0.87
second actuator	command language → half-scale, 45°, re-chunked (6-d vs A's 10-d); same pixels, B relabels A	the frozen dictionary is reused for B: effect-head cos 0.974, control 58.7 vs 58.0, atom identity cos 0.995	same-observation cross-actuation (B's data relabels A's), not cross-morphology
scene instance	maze layout → never seen	action fit 0.26→0.66; 91% retained across held-out layouts; interaction residual localizes walls (three-seed AUC 0.943, label-free)	instance-invariance on the data manifold (0.73 on, 0.36 off)
dynamics	physics → damping / gain×0.5 / off-center mass	controlled-replay concentration score η^2_{raw} remains 0.65 under the shift; a closed-form refit restores 81%	requires recalibration; zero-shot transfer fails for gain shifts

D TRANSFER AND ROBUSTNESS ANALYSES

D.1 COMMAND-INTERFACE TRANSFER

Actuation-interface construction. We keep the encoder and dictionary fixed and fit separate heads for each actuation interface e. An interface-specific effect head ρ_e maps commands to action prototypes, and an inverse head g_e maps consecutive latent states back to commands. This lets interfaces with different command coordinates refer to the same action prototypes.

Evaluation design. We vary four components separately: the visual backbone, the actuation interface, the scene layout, and the dynamics. For each comparison, the other components remain fixed. Table 9 summarizes the results; the following subsections collect the corresponding evidence.

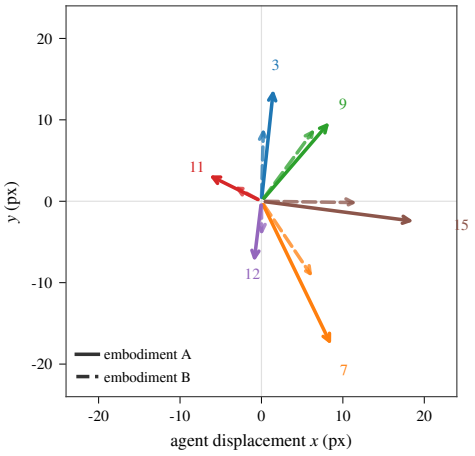


Figure 15: **Atom identity across actuation interfaces.** Mean world-frame motion direction of each atom under interface A (solid) and B (dashed): directions agree at cos 0.995, magnitudes scale by chunk duration. The atom names the motion; the actuation interface determines the command.

The axis-only dictionary also transfers to interface B: on the 150-task evaluation, it reaches 59.3%, matching the full-data arms although the dictionary and interface-B heads were fitted without world-diagonal motions. This changes the command parameterization with fixed observations, not the agent's morphology.

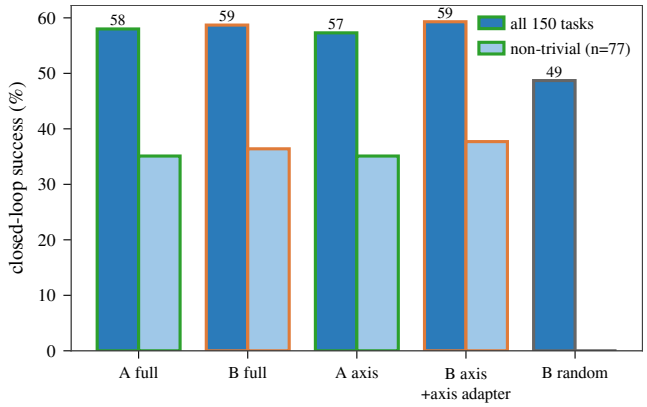


Figure 16: **One frozen dictionary, two actuation interfaces** (greedy decomposition-following; light bars restrict to the stratum where random commands fail). B matches A at parity, and the axis-only arm — whose dictionary and interface-B heads never saw a world-diagonal motion — matches the full arms.

D.2 Frozen visual features

Table 10 and Figure 17 compare raw DINOv2 features with a trained two-layer projector, with and without the concentration loss. The encoder remains frozen throughout.

Table 10: **Transition concentration on frozen DINOv2 features** (Push-T; only a two-layer projector and the heads are trained). Raw frozen latent transitions are action-blind although the features contain the state; the concentration loss $\mathcal{L}_{\text{equiv}}$ recovers most of the end-to-end structure through the projector alone.

features	trained part	$R^2(a \mapsto \Delta z)$ lin/MLP	$\text{block}_{x/y}$ (MLP)
patch, pooled	none (raw)	0.02 / −0.06	.90 / .96
patch, pooled	projector, λ_{eq}=0	0.38 / 0.29	.82 / .88
patch, pooled	projector, λ_{eq}=10	**0.70 / 0.65**	.77 / .90
CLS	projector, λ_{eq}=10	0.27 / 0.20	.69 / .90
end-to-end reference (λ_{eq}=10)		0.86 / 0.87	.93 / .92

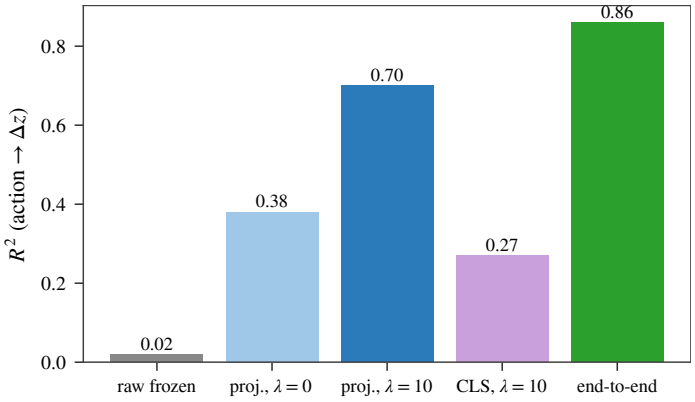


Figure 17: **Frozen DINOv2 post-training** (companion to Table 10). Raw frozen latent transitions are action-blind; a two-layer projector with the concentration loss recovers most of the end-to-end structure.

D.3 UNSEEN MAZE LAYOUTS

On Maze, action fit increases from 0.26 to 0.66 and retains 91% of its value on unseen layouts. Section 5.2 evaluates wall localization from the interaction residual. Off the training-state distribution, action fit drops from 0.73 to 0.36.

D.4 DYNAMICS SHIFTS AND READOUT RECALIBRATION

Table 11 separates zero-shot performance from recalibration. A closed-form refit of the 10×192 action readout recovers 81% of the original action fit under the gain shift without updating the encoder. Unlike test-time adaptation (Wang et al., 2026a), this calibration changes only the linear readout.

Table 11: **Dynamics shifts on Push-T** (frozen encoders, controlled rollouts). Zero-shot: $\hat{\rho}$ fitted on default dynamics; refit: $\hat{\rho}$ re-estimated on half the probe states and evaluated on the rest. Dict. FVU uses the frozen SAE of each latent space. The controlled-replay concentration score η^2_{raw} is reported overall and on free-space transitions in parentheses; the next two columns report zero-shot and refit action fits.

model	shift	η^2_{raw} (free)	zero-shot $R^2(a \mapsto \Delta z)$	refit $R^2(a \mapsto \Delta z)$	dict. FVU
EquiJEPA	none	0.653 (0.645)	0.649	0.646	0.161
EquiJEPA	damping	0.611 (0.628)	0.605	0.591	0.198
EquiJEPA	gain $\times0.5$	0.599 (0.728)	0.089	**0.525**	0.273
EquiJEPA	block mass off-center	0.646 (0.649)	0.642	0.640	0.168
base	none	0.461 (0.521)	0.453	0.449	0.258
base	gain $\times0.5$	0.376 (0.575)	0.051	0.323	0.402

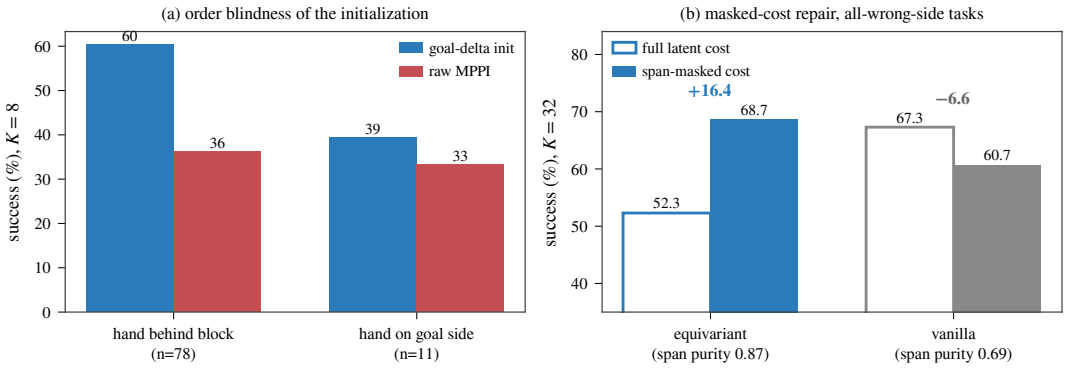


Figure 18: **Planning when the hand starts on the goal side of the block. (a)** The benefit of goal-delta initialization is smaller when the plan must initially move away from the goal. **(b)** Removing the action-controlled component from the planning cost improves success by 16.4 points within EquiJEPA. Relative to the best base cost, the gain is 1.4 points.

E Planning evaluations and controls

E.1 Initialization and dictionary projection

Planning setup. Goal-conditioned MPC rolls the predictor forward and scores each candidate action sequence by its terminal latent distance to the goal (Zhou et al., 2025). Given current and goal latents, let $\Delta^* = z_g - z_t$ denote the desired change. Here we study its use as an MPPI initialization (Williams et al., 2016), with and without dictionary projection.

Goal-delta initialization. The direct variant places H equally spaced waypoints along the raw goal delta,

$$w_h = z_t + \frac{h}{H}\Delta^*, \qquad h = 0, \dots, H,$$

and decodes each consecutive pair using the inverse-dynamics head, $\hat{a}_h = g_\omega(w_h, w_{h+1})$. The resulting action sequence initializes the mean of an otherwise unchanged MPPI solver. This is a straight-line prior that does not require the dictionary, so its value depends on the task geometry. Because MPPI updates this mean, the initialization informs all solver iterations (Wang et al., 2026b).

Optional dictionary projection. Alternatively, we project the goal delta onto combinations supported by the learned dictionary:

$$c^* = \arg\min_{c \geq 0} \left\| (\Delta^* - Hb) - c^\top \mathbf{D} \right\|_2^2, \tag{9}$$

where H is the planning horizon in action chunks and b is the per-transition dictionary bias. Projected gradient descent solves this small nonnegative least-squares problem. The reconstructed change is $\widetilde{\Delta}_D = c^{*\top}\mathbf{D} + Hb$, with waypoints

$$w_h = z_t + \frac{h}{H} c^{*\top}\mathbf{D} + hb.$$

This projection removes components the action dictionary cannot express. We evaluate raw and projected deltas separately, so the contribution of decomposition is not conflated with that of initialization. In either case, the sequence is recomputed after every environment step.

Proposition 3 in Appendix F bounds the accumulation of interaction and goal-decomposition errors.

E.2 Evaluation protocol

We compare raw and dictionary-projected goal deltas on both the EquiJEPA and base representations. Push-T results use paired tasks and separately calibrated MPPI temperatures. We repeat the comparison on Cube and Maze to test how it depends on task geometry (Table 13).

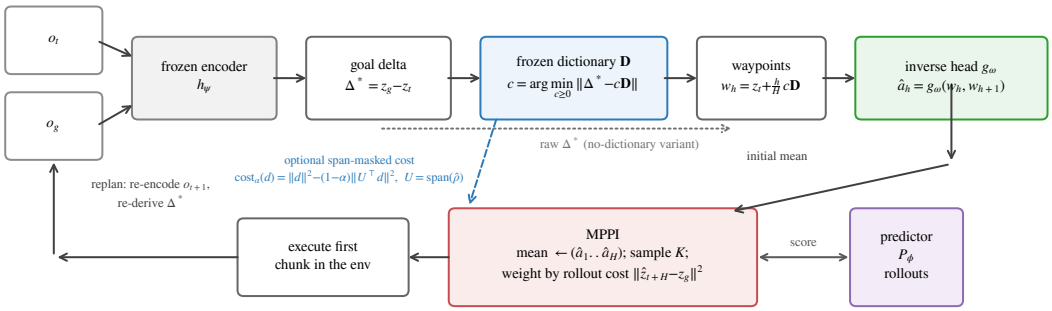


Figure 19: **The inference-time loop.** The frozen encoder maps current and goal observations to latents; the goal delta is decomposed over the frozen dictionary; waypoints are laid along the reconstruction and the inverse head reads each transition into an action chunk, initializing the mean of an otherwise unmodified MPPI solver. Dotted: the no-dictionary attribution arm (Table 15); dashed blue: the optional action-subspace cost used for the wrong-side repair (Sec. E.3).

Cube and maze use 50 tasks with per-arm temperatures calibrated at $K{=}32$ on a disjoint task seed. Each task starts from an expert state. On Cube the goal is the expert's block position 25 steps ahead, success is the block within 4 cm, and tasks are filtered to a block displacement of at least 6 cm. On Maze the goal is the expert's agent position 12 steps ahead with the environment's native goal-radius success criterion and a start-to-goal displacement of at least 0.8. Maze MPPI uses noise scale 0.2, because the solver's noise is expressed in raw action units and maze actions span ± 0.2.

Table 12 reports goal-delta initialization for EquiJEPA; LeWM-style has no inverse head with which to decode the waypoints. DINO-WM-style is absent because its frozen features provide no predictor for rolling out action candidates.

E.3 Planning results

Summary. Planning uses a low-budget protocol ($K{\leq}32$), so absolute success is not comparable to the budgets used in LeWM (Maes et al., 2026b); success on Push-T also varies strongly with the evaluation protocol (Maes et al., 2026a). The goal-delta initialization helps both representations when a straight latent path is feasible. Under the same initialization at $K{=}8$, EquiJEPA is 10.7 points higher than the base on Cube and in Maze and 4.2 points lower on Push-T (Table 12). Without search ($K{=}1$), the decoded plans of the two models succeed equally often on Push-T (45.3 versus 46.0%) and Cube (38.0% each) and EquiJEPA's more often in the maze (54.0 versus 46.0%), so on Cube its advantage appears only once MPPI refines the initialization. In Maze, the initialization lowers success for both models because straight waypoints may cross walls.

Where the Push-T gap sits. We split the 150 Push-T tasks into terciles of the block rotation required to reach the goal (edges 5.1° and 18.8°) and, separately, of the block translation (edges 11.7 and 48.1 px). With the same initialization, the base leads in the low- and mid-rotation terciles (80.0 versus 74.7 and 65.3 versus 54.7 at $K{=}8$) and the two are level in the high-rotation tercile (61.3 versus 64.7); across translation terciles the base leads by 2–9 points throughout. The gap is therefore not specific to rotation-heavy tasks, and we do not attribute it to a particular task geometry.

Standardized cost. With a per-dimension standardized cost, raw MPPI on EquiJEPA gains 7.7, 7.7 and 11.8 points at $K{=}8$, 16 and 32, against 2.6, 1.3 and 2.5 for the base; the gap between the two representations under raw MPPI shrinks from 9–14 to 3–9 points. EquiJEPA's latent dimensions differ in scale by 7.4× (5.5× for the base), so a few dimensions dominate the unweighted cost.

Sample efficiency. Table 12 reports the small-budget regime, where an initialization matters. On Push-T the gain over the same representation is +20.0, +15.3 and +17.1 points at $K{=}8$, 16 and 32; extending the sweep to $K{=}128$ shrinks it to +4.9, since a large enough search finds the same

Table 12: **Full planning sweep** (model seed 0; mean ± SD over three solver seeds; Push-T 150 tasks per seed, Cube and Maze 50). "std" scores candidates with a per-dimension standardized cost $\|\mathrm{diag}(1/\sigma)\, d\|^2$, where σ_j is the standard deviation of latent dimension j; its temperature is recalibrated. At $K{=}1$ the initialized planner executes the decoded plan without search and is deterministic across solver seeds. Bold marks the best result in each row.

	K	EquiJEPA + init	base + init	EquiJEPA + MPPI	EquiJEPA + MPPI (std)	base + MPPI	base + MPPI (std)	LeWM + MPPI
Push-T	1	45.3	**46.0**	–	–	–	–	–
	8	64.7 ± 2.9	$\mathbf{68.9 \pm 1.9}$	44.7 ± 3.5	52.4 ± 2.8	58.7 ± 5.3	61.3 ± 5.9	52.9 ± 1.7
	16	66.9 ± 2.3	$\mathbf{70.2 \pm 2.5}$	51.6 ± 2.3	59.3 ± 3.7	60.7 ± 2.9	62.0 ± 2.4	60.7 ± 4.4
	32	71.1 ± 3.7	$\mathbf{75.8 \pm 0.8}$	54.0 ± 1.8	65.8 ± 2.3	67.3 ± 1.8	69.8 ± 3.4	61.6 ± 3.7
Cube	1	**38.0**	**38.0**	–	–	–	–	–
	8	$\mathbf{46.0 \pm 8.7}$	35.3 ± 7.0	12.7 ± 4.2	–	13.3 ± 3.1	–	10.7 ± 2.3
	16	$\mathbf{43.3 \pm 3.1}$	32.0 ± 3.5	13.3 ± 4.2	–	14.0 ± 7.2	–	15.3 ± 4.6
	32	$\mathbf{44.7 \pm 1.2}$	34.0 ± 2.0	14.0 ± 5.3	–	19.3 ± 7.0	–	16.7 ± 2.3
Maze	1	**54.0**	46.0	–	–	–	–	–
	8	62.7 ± 1.2	52.0 ± 3.5	$\mathbf{73.3 \pm 3.1}$	–	69.3 ± 1.2	–	14.7 ± 1.2
	16	60.7 ± 3.1	48.7 ± 8.1	$\mathbf{74.0 \pm 0.0}$	–	69.3 ± 4.2	–	14.7 ± 4.2
	32	59.3 ± 4.2	52.0 ± 5.3	68.7 ± 3.1	–	$\mathbf{70.7 \pm 6.4}$	–	8.7 ± 1.2

Table 13: **Goal-delta initialization across environments** ($K{=}8$). Each arrow compares raw MPPI with initialized MPPI on the same representation. Push-T uses the attribution protocol in Table 15; Cube and maze report three solver-seed sweeps for model seed 0 (Table 12). The three-model-seed Cube replication is in Table 14.

environment (geometry)	EquiJEPA: raw → init	base: raw → init	EquiJEPA − base, both init
Push-T (open arena)	$42.0 \to 68.0$ $(+26.0)$	$53.3 \to 72.7$ $(+19.4)$	-4.7
Cube (sustained grasp)	$12.7 \to 46.0$ $(+33.3)$	$13.3 \to 35.3$ $(+22.0)$	$+10.7$
Maze (corridors)	$73.3 \to 62.7$ (-10.6)	$69.3 \to 52.0$ (-17.3)	$+10.7$

solutions unaided. Initialized EquiJEPA is 4–6 points above the base with raw MPPI at $K \leq 32$ and 3–5 points below the base with the same initialization. On Cube the gain is far larger and does not fade ($12.7 \to 46.0$ at $K{=}8$). In Maze the initialization instead costs 9–13 points for EquiJEPA. LeWM-style separates the two failure modes: it is competitive on Push-T (53–62) yet collapses in Maze (9–15), so a predictor without action information can still be steered by cost weighting in an open arena but not where the goal must be reached precisely. We calibrate temperatures separately because candidate costs have an $8\times$ smaller spread in the EquiJEPA latent. A temperature-free CEM (Rubinstein, 1999) control gives the same ordering (Appendix Table 16).

Raw versus projected goal deltas. Appendix Table 15 compares three variants. On in-distribution Push-T goals, waypoints along the raw goal delta perform as well as waypoints along the dictionary projection. The same raw-delta initialization also improves the base model, so the main sample-efficiency gain comes from initialization rather than dictionary decomposition. The projection becomes useful when the goal latent is heavily corrupted, where it removes components outside the dictionary, but it changes in-distribution goal directions very little. Direct search over dictionary coefficients performs worse because it restricts proposals to straight paths in latent space.

The learned inverse head is important for decoding the waypoints. Pseudo-inverting a linear action-to-transition map gives an action cosine of 0.09, compared with 0.97 for the inverse head. The linear inverse is unstable because the expert action chunks have low effective rank and the goal delta contains components unrelated to action.

Dependence on task geometry. Table 13 repeats the comparison on Cube and the maze. On Cube, zero-mean samples in the 25-dimensional action-chunk space rarely maintain a grasp. Raw MPPI therefore remains at 13–20% success, while initialization raises success to 46.0% for EquiJEPA and 35.3% for the base model at $K{=}8$ in model seed 0. Across three model seeds the comparison is 46.7% versus 35.3% (Table 14). The 10–11 point difference between the initialized models persists for $K \leq 32$. Cube also has the largest action-determined fraction of Δz (0.95 versus 0.82), and success is measured with a 4 cm tolerance, making waypoint decoding accuracy important.

Table 14: **Cube planning over three model seeds.** Success (%) at $K=8$ on sustained-grasp tasks. Variation is across model seeds after task/solver aggregation; the base has identical aggregated success across its three seeds. Maze wall-localization results are reported in Section 5.2.

representation	Cube planning ↑
base	35.3 ± 0.0
EquiJEPA	$\mathbf{46.7 \pm 1.2}$

In Maze, a straight path to the goal often crosses a wall. The initialization reduces success by 10.6 points on EquiJEPA and 17.3 points on the base model at $K=8$, with larger reductions at higher budgets. These results show that goal-delta initialization is useful when a straight latent path is compatible with the task, as in Push-T and Cube, but not in constrained corridor geometry.

E.4 Results across model seeds

E.5 Initialization attribution and solver controls

Table 15: **Attribution: what causes the planning gain.** Same 150 paired tasks, calibrated temperatures, one solver seed; each cell reports success (%) at $K=8$ / $K=32$. The goal-delta initialization is model-agnostic and causal; the dictionary projection leaves it essentially unchanged (its reconstruction retains the direction of the raw delta at a cosine of 0.97); searching in the coefficient space itself underperforms.

initializer / search	base	$\lambda_{\text{eq}}=10$
raw MPPI (shifted-plan warm start)	53.3 / 66.0	42.0 / 54.7
goal-delta init, no dictionary	**72.7 / 78.7**	68.0 / 75.3
goal-delta init, dictionary projection	70.7 / 74.0	**68.0 / 75.3**
atom-coefficient search (coefficient cone)	—	47.3 / 48.7

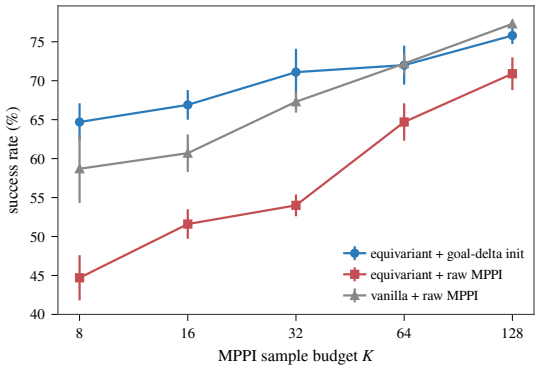


Figure 20: **Success versus sample budget** (curves for Table 12; mean±std over three solver seeds). The initialized planner at $K=8$ matches raw MPPI at $K=64$.

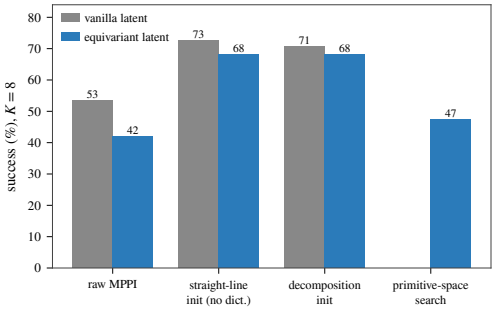


Figure 21: **Attribution arms** (bars for Table 15). The straight-line initializer matches or beats the decomposition on both latents; coefficient-space search underperforms.

E.6 Physical and planning limits

On Push-T, ground-truth action fit is only 0.554 because contact makes object motion state dependent. The learned dictionary primarily captures hand motion; block-only displacement has a much larger dictionary reconstruction residual. EquiJEPA does not outperform the base in temperature-free Push-T CEM. In Maze, straight latent waypoints can cross walls and hurt both models. These failures support the prototype–interaction-residual decomposition: concentration supplies a reusable

Table 16: **Per-K CEM control** (protocol of Table 12: 150 paired episodes, three solver seeds, elite fraction $K/4$). CEM has no temperature, so it is immune to the calibration concern by construction — and it reproduces the ordering of the calibrated MPPI arms: the raw base latent searches somewhat better than the raw EquiJEPA latent, consistent with the attribution finding, while the initialized EquiJEPA arm of Table 12 (64.7 at K=8) outperforms both CEM arms at small budgets.

K	EquiJEPA + CEM	base + CEM
8	40.7 ± 1.9	44.7 ± 0.9
16	52.0 ± 5.7	59.3 ± 5.0
32	60.7 ± 5.7	72.0 ± 1.6
64	68.0 ± 1.6	76.7 ± 4.1
128	71.3 ± 3.8	82.0 ± 2.8

latent action geometry where the task geometry permits it, without asserting exact state-independent physics or universal planning improvement. The preceding controls distinguish initialization gains from differences between representations.

Solver hyperparameters, full per-dimension probe tables, the temperature-calibration sweep, and wall-clock figures accompany the released analysis toolkit.

F THEORETICAL RESULTS AND PROOFS

Throughout, actions act on states through maps T_a and $\Delta z(s,a) = h_\psi(T_a s) - h_\psi(s)$.

Proposition 2 (linearity of ρ; cf. Thm. 1 of Ivashkov et al., 2026)**.** *If the encoder is translation-equivariant and actions compose additively ($T_b \circ T_a = T_{a+b}$), then $\rho(a+b) = \rho(a) + \rho(b)$; if ρ is measurable, it is linear, $\rho(a) = Wa$.*

F.1 PROOF OF PROPOSITION 2

For any state s, equivariance applied to $T_{a+b}\, s$ gives $h_\psi(T_{a+b}\, s) = h_\psi(s) + \rho(a+b)$, while applying it twice along $T_b(T_a s)$ gives $h_\psi(T_b(T_a s)) = h_\psi(T_a s) + \rho(b) = h_\psi(s) + \rho(a) + \rho(b)$. Since $T_{a+b} = T_b \circ T_a$, the left-hand sides coincide, and therefore $\rho(a+b) = \rho(a) + \rho(b)$ for all a, b, with $\rho(0) = 0$. Each coordinate of ρ is then an additive function $\mathbb{R}^m \to \mathbb{R}$; additivity implies $\mathbb{Q}$-homogeneity, and a measurable additive function is linear by the classical theorem of Banach and Sierpiński on the Cauchy functional equation (Banach, 1920; Sierpiński, 1920). Non-measurable additive functions exist only through Hamel-basis constructions and cannot be realized by a continuous model such as an MLP, so measurability is not a practical restriction. □

The practical content is this: whenever the environment composes additively and the encoder is equivariant, the closed form of ρ is a single matrix, and a nonlinear ρ can at best re-implement it. Training with a linear ρ gives the lowest multi-step composition error on Push-T (Appendix B.2).

F.2 PROOF OF PROPOSITION 1

(i) Under equivariance, $\Delta z(s,a) = \rho(a)$ does not depend on s, so $\mathrm{Var}_s[\Delta z \mid a] = 0$, and injectivity of ρ makes $D = \rho^{-1}$, defined on the range of ρ, an exact decoder.

(ii) Let $S = \mathbb{R}^m \times \mathbb{R}^k$ with states $s = (x, y)$, transitions $T_a(x, y) = (x + a,\ g(x, y))$ for an arbitrary map g, and the identity encoder. Then

$$\Delta z(s,a) = \big(a,\ g(x,y) - y\big).$$

The projection onto the first block decodes a exactly, for every state and action. The second block does not depend on a and can be given arbitrarily large variance over states by scaling g, so $\inf_a \mathrm{Var}_s[\Delta z \mid a] \geq c$ for any prescribed c. □

The construction mirrors manipulation: the first block plays the end effector, which follows the action, and the second block plays the object, which moves as a function of state. A decoder that reads the action from Δz constrains only the first block.

F.3 MULTI-STEP COMPOSITION: STATEMENT AND PROOF

Interaction residuals and imperfect goal decomposition accumulate as follows:

Proposition 3 (composition and decomposition error)**.** *Let $e_t = \Delta z_t - \rho(a_t)$ with $\|e_t\| \leq \varepsilon$ for all t. (i) For every horizon k, $\|(z_{t+k} - z_t) - \sum_{i<k} \rho(a_{t+i})\| \leq k\varepsilon$; if the e_t are zero-mean and uncorrelated, the expected squared error grows only as $k\sigma^2$. (ii) If $\Delta^* = \sum_{i<k} \rho(\hat{a}_i) + r_{\mathrm{goal}}$, executing the decoded actions lands within $k\varepsilon + \|r_{\mathrm{goal}}\|$ of the goal in latent space.*

Here $e_t = r_{\mathrm{act}}(s_t, a_t)$ is the interaction residual, ε bounds its one-step magnitude, and r_{goal} is the part of the goal change left outside the action dictionary. Part (i) shows that controlling one-step action-prototype error also controls temporal-composition error, without an additional multi-step objective.

Proof. (i) Telescoping the latent transitions gives

$$z_{t+k} - z_t = \sum_{i<k} \Delta z_{t+i} = \sum_{i<k} \rho(a_{t+i}) + \sum_{i<k} e_{t+i},$$

and the triangle inequality bounds the interaction-residual sum by $k\varepsilon$. If the e_t are zero-mean and pairwise uncorrelated, then $\mathbb{E}\big\|\sum_{i<k} e_{t+i}\big\|^2 = \sum_{i<k} \mathbb{E}\|e_{t+i}\|^2 = k\sigma^2$, since the cross terms vanish.

(ii) Subtracting the goal decomposition $\Delta^* = z_g - z_t = \sum_{i<k} \rho(\hat{a}_i) + r_{\text{goal}}$ from the telescoped sum under the executed actions $\hat{a}_i$ yields $z_{t+k} - z_g = \sum_{i<k} e_{t+i} - r_{\text{goal}}$, and the triangle inequality gives the claim. □

Two remarks. First, the executed actions are produced by the inverse-head readout rather than by an exact inverse of ρ, which adds a per-step readout error and turns the bound into $k(\varepsilon+\delta)+\|r_{\text{goal}}\|$; the readout is accurate on the synthetic waypoint pairs the planner queries (cosine 0.91 against ground-truth actions), so δ is small in practice. Second, the bound of part (i) is loose in our favor on real data: the measured relative composition error *decreases* with horizon (from 0.49 at one chunk to 0.31 at three on Push-T), which indicates that consecutive interaction residuals anti-correlate rather than accumulate.